

MinMaxScaler#

<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

Description:

Transform features by scaling each feature to a given range. This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one. The transformation is given by:
$$\text{feature_range} = \frac{\text{feature} - \text{min}}{\text{max} - \text{min}}$$
 where min, max = feature_range.

Function Signature

```
class sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1), *, copy=True, clip=False)[source]#
```

Parameters:

feature_range tuple (min, max), default=(0, 1)

copy bool, default=True

clip bool, default=False

```
class sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1), *, copy=True, clip=False)[source]#
```

Parameters:

feature_range tuple (min, max), default=(0, 1)

copy bool, default=True

clip bool, default=False

fit(X, y=None)[source]#

Parameters:

X array-like of shape (n_samples, n_features)

Returns:

Parameters

`Xarray`

The data used to compute the per-feature minimum and maximum used for later scaling along the features axis.

`Xarray`

Input samples.

`yarray`

Target values (None for unsupervised transformations).

`**fit_paramsdict`

Additional fit parameters.

`input_featuresarray`

Input features. If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`. If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

`paramsdict`

Parameter names mapped to their values.

`Xarray`

Input data that will be transformed. It cannot be sparse.

`Xarray`

The data used to compute the mean and standard deviation used for later scaling along the features axis.

`**paramsdict`

Estimator parameters.

`Xarray`

Input data that will be transformed.

Code Examples

```
X_std = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
X_scaled = X_std * (max - min) + min
```

```
>>> from sklearn.preprocessing import MinMaxScaler
>>> data = [[-1, 2], [-0.5, 6], [0, 10], [1, 18]]
>>> scaler = MinMaxScaler()
>>> print(scaler.fit(data))
MinMaxScaler()
>>> print(scaler.data_max_)
[ 1. 18.]
>>> print(scaler.transform(data))
[[0.  0. ]
 [0.25 0.25]
 [0.5  0.5 ]
 [1.   1.  ]]
>>> print(scaler.transform([[2, 2]]))
[[1.5 0.  ]]
```

Notes & Warnings

NOTE

See also `minmax_scale` Equivalent function without the estimator API.

Detailed Documentation

Documentation

Transform features by scaling each feature to a given range.

This estimator scales and translates each feature individually such that it is in the given range on the training set, e.g. between zero and one.

The transformation is given by:

where $\min$, $\max$ = `feature_range`.

This transformation is often used as an alternative to zero mean, unit variance scaling.

`MinMaxScaler` doesn't reduce the effect of outliers, but it linearly scales them down into a fixed range, where the largest occurring data point corresponds to the maximum value and the smallest one corresponds to the minimum value. For an example visualization, refer to [Compare MinMaxScaler with other scalers](#).

Read more in the [User Guide](#).

Desired range of transformed data.

Set to `False` to perform inplace row normalization and avoid a copy (if the input is already a numpy array).

Set to `True` to clip transformed values of held-out data to provided feature range.

Added in version 0.24.

Per feature adjustment for minimum. Equivalent to $\min - X.\min(\text{axis}=0) * \text{self.scale_}$

Per feature relative scaling of the data. Equivalent to $(\max - \min) / (X.\max(\text{axis}=0) - X.\min(\text{axis}=0))$

Added in version 0.17: `scale_` attribute.

Per feature minimum seen in the data

Added in version 0.17: `data_min_`

Per feature maximum seen in the data

Added in version 0.17: `data_max_`

Per feature range (`data_max_ - data_min_`) seen in the data

Added in version 0.17: `data_range_`

Number of features seen during fit.

Added in version 0.24.

The number of samples processed by the estimator. It will be reset on new calls to `fit`, but increments across `partial_fit` calls.

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.

Equivalent function without the estimator API.

NaNs are treated as missing values: disregarded in fit, and maintained in transform.

Compute the minimum and maximum to be used for later scaling.

The data used to compute the per-feature minimum and maximum used for later scaling along the features axis.

Fit to data, then transform it.

Fits transformer to `X` and `y` with optional parameters `fit_params` and returns a transformed version of `X`.

Target values (None for unsupervised transformations).

Additional fit parameters.

Get output feature names for transformation.

If `input_features` is `None`, then `feature_names_in_` is used as feature names in. If `feature_names_in_` is not defined, then the following input feature names are generated: `["x0", "x1", ..., "x(n_features_in_ - 1)"]`.

If `input_features` is an array-like, then `input_features` must match `feature_names_in_` if `feature_names_in_` is defined.

Same as input features.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A `MetadataRequest` encapsulating routing information.

Get parameters for this estimator.

If `True`, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Undo the scaling of X according to feature_range.

Input data that will be transformed. It cannot be sparse.

Online computation of min and max on X for later scaling.

All of X is processed as a single batch. This is intended for cases when fit is not feasible due to very large number of n_samples or because X is read from a continuous stream.

The data used to compute the mean and standard deviation used for later scaling along the features axis.

Set output container.

See [Introducing the set_output API](#) for an example on how to use the API.

Configure output of transform and fit_transform.

"default": Default output format of a transformer

"pandas": DataFrame output

"polars": Polars output

None: Transform configuration is unchanged

Added in version 1.4: "polars" option was added.

Set the parameters of this estimator.

The method works on simple estimators as well as on nested objects (such as Pipeline). The latter have parameters of the form <component>__<parameter> so that it's possible to update each component of a nested object.

Estimator parameters.

Scale features of X according to feature_range.

Input data that will be transformed.

Time-related feature engineering

Image denoising using kernel PCA

Selecting dimensionality reduction with Pipeline and GridSearchCV

Univariate Feature Selection

Recursive feature elimination

Scalable learning with polynomial kernel approximation

Manifold learning on handwritten digits: Locally Linear Embedding, Isomap...

Evaluation of outlier detection estimators

Compare Stochastic learning strategies for MLPClassifier

Compare the effect of different scalers on data with outliers

Release Highlights for scikit-learn 0.24

© Copyright 2007 - 2025, scikit-learn developers (BSD License).